SENA

SERVICIO NACIONAL DE APRENDIZAJE

Centro de Teleinformática y Producción Industrial • Regional Cauca

EVIDENCIA GA7-220501096-AA2-EV01

Informe Técnico de Construcción: Codificación de Módulos del Software con Java JDBC y Arquitectura DAO

Implementación del módulo backend de persistencia del sistema Pizzería Bella Italia mediante Java 17, controladores JDBC parametrizados, transacciones ACID y orquestación ligera con Docker Compose.

APRENDIZ:

Karen Yuliana Campo
Correo: karen.campo@sena.edu.co
Ubicación: Popayán, Cauca (100% Virtual)

PROGRAMA DE FORMACIÓN:

Análisis y Desarrollo de Software (ADSO)
Ficha de Caracterización: 228118
Fecha: Noviembre 2026

1. Introducción y Justificación Técnica

En la fase de ejecución del programa formativo ADSO (Actividad de Proyecto 7: Codificar los módulos del software), el componente de acceso a datos constituye el cimiento de confiabilidad de cualquier solución de software. La tecnología **Java Database Connectivity (JDBC)** ofrece una interfaz de bajo nivel de alto rendimiento que conecta el lenguaje Java con sistemas relacionales basados en SQL.

La presente evidencia, codificada bajo la referencia **GA7-220501096-AA2-EV01**, traslada el diseño físico de bases de datos formulado en fases anteriores (archivo `GA6-220501096-AA2-EV03.tsql`) a una implementación funcional orientada a objetos para el sistema de la **Pizzería Bella Italia**. Se aplica el patrón de diseño **DAO (Data Access Object)** y la separación de capas (Modelo, Utilidades, DAO y Aplicación), mitigando el acoplamiento y previniendo vulnerabilidades como la inyección de código SQL.

> **Resumen de Características Técnicas Desarrolladas:**
> • Lenguaje: Java 17 LTS (Eclipse Temurin JDK) gestionado con Apache Maven 3.9.
> • RDBMS: MariaDB 10.5 / MySQL 8.0 con motor transaccional InnoDB.
> • Entidades: Clientes, Pizzas (Catálogo), Pedidos (Cabecera) y DetallePedido (Líneas de venta).
> • Operaciones: CRUD completas (Create, Read, Update, Delete) + Transacciones atómicas ACID.
> • Orquestación: Docker Compose modular (servicios `db`, `backend-java` y `test-runner`).

2. Estándares de Codificación Aplicados

Nivel	Estándar Exigido	Implementación en el Proyecto	Finalidad Técnica
Paquetes	Dominio inverso en minúsculas	`com.sena.pizzeria.model` `com.sena.pizzeria.dao` `com.sena.pizzeria.util`	Aislamiento de namespaces y modularidad estructural.
Clases	PascalCase (Sustantivos)	`Cliente`, `Pizza`, `Pedido`, `ClienteDAO`	Claridad semántica e interoperabilidad con JavaBeans.
Métodos	camelCase (Verbo + Objeto)	`insertar()`, `consultarPorId()`, `listarTodos()`	Expresión inequívoca de acción y responsabilidad única (SRP).
Variables	camelCase descriptivo	`idCliente`, `nombrePizza`, `filasAfectadas`	Mantenibilidad y legibilidad limpia de código (Clean Code).

3. Arquitectura del Proyecto Maven (`PizzeriaBackend`)

```
PizzeriaBackend/
├── pom.xml                                  (Dependencias Java 17, MariaDB Driver y JUnit 5)
└── src/
    ├── main/java/com/sena/pizzeria/
    │   ├── model/                           (Capa de Entidades POO)
    │   │   ├── Cliente.java                 (Encapsulamiento de cliente y datos de contacto)
    │   │   ├── Pizza.java                   (Catálogo de productos, tamaño y precio)
    │   │   ├── Pedido.java                  (Orden de compra, fecha, total y estado)
    │   │   └── DetallePedido.java           (Ítem con pizza, cantidad y subtotal)
    │   ├── util/                            (Infraestructura de Conexión)
    │   │   └── ConexionBD.java              (Gestor JDBC Singleton con variables de entorno)
    │   ├── dao/                             (Patrón Data Access Object)
    │   │   ├── ClienteDAO.java              (CRUD Clientes con PreparedStatement)
    │   │   ├── PizzaDAO.java                (CRUD Pizzas con PreparedStatement)
    │   │   └── PedidoDAO.java               (CRUD Pedidos y Transacciones ACID)
    │   └── app/                             (Punto de Entrada)
    │       └── Main.java                    (Demostración en vivo de operaciones CRUD)
    └── test/java/com/sena/pizzeria/
        └── PizzeriaCRUDTest.java            (Suite de Pruebas Unitarias JUnit 5)
```

4. Implementación del Acceso a Datos (JDBC & CRUD)

4.1. Conexión Parametrizada y Resiliente (`ConexionBD.java`)

```
public static Connection getConexion() throws SQLException {
    String host = System.getenv("DB_HOST") != null ? System.getenv("DB_HOST") : "localhost";
    String port = System.getenv("DB_PORT") != null ? System.getenv("DB_PORT") : "3306";
    String database = System.getenv("DB_NAME") != null ? System.getenv("DB_NAME") : "pizzeria";
    String user = System.getenv("DB_USER") != null ? System.getenv("DB_USER") : "karen";
    String password = System.getenv("DB_PASS") != null ? System.getenv("DB_PASS") : "choco";

    String url = String.format("jdbc:mariadb://%s:%s/%s?useSSL=false&allowPublicKeyRetrieval=true",
            host, port, database);
    return DriverManager.getConnection(url, user, password);
}
```

4.2. Inserción Segura y Generación de Llaves (`ClienteDAO.java`)

```
public boolean insertar(Cliente cliente) {
    String sql = "INSERT INTO CLIENTES (Nombre, Telefono, Direccion) VALUES (?, ?, ?)";
    try (Connection conn = ConexionBD.getConexion();
         PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        stmt.setString(1, cliente.getNombre());
        stmt.setString(2, cliente.getTelefono());
        stmt.setString(3, cliente.getDireccion());
        if (stmt.executeUpdate() > 0) {
            try (ResultSet rs = stmt.getGeneratedKeys()) {
                if (rs.next()) cliente.setIdCliente(rs.getInt(1));
            }
            return true;
        }
    } catch (SQLException e) { System.err.println(e.getMessage()); }
    return false;
}
```

4.3. Transaccionalidad Atómica ACID (`PedidoDAO.java`)

Para asegurar que un pedido se registre únicamente si todos sus ítems de detalle son almacenados con éxito, se desactiva el *auto-commit* y se gestionan las confirmaciones mediante `commit()` o retroceso con `rollback()` ante cualquier anomalía:

```
conn.setAutoCommit(false); // Inicia transacción atómica
// 1. Inserción en tabla PEDIDOS
stmtPedido.executeUpdate();
int idPedido = obtenerIdGenerado(stmtPedido);

// 2. Inserción en batch sobre DETALLE_PEDIDO
for (DetallePedido item : pedido.getDetalles()) {
    stmtDetalle.setInt(1, item.getCantidad());
    stmtDetalle.setDouble(2, item.getSubtotal());
    stmtDetalle.setInt(3, idPedido);
    stmtDetalle.setInt(4, item.getIdPizza());
    stmtDetalle.addBatch();
}
stmtDetalle.executeBatch();
conn.commit(); // Confirmación atómica si todo fue exitoso
```

5. Resultados de Pruebas Unitarias y Ejecución en Consola

5.1. Resultados JUnit 5 (`PizzeriaCRUDTest.java`)

Caso de Prueba	Objetivo Verificado	Resultado
`testConexionBD()`	Valida conectividad activa con MariaDB 10.5 a través del driver JDBC.	PASÓ (0.51s)
`testClienteCRUD()`	Verifica ciclo completo: Inserción, Consulta por ID, Actualización, Listado y Eliminación.	PASÓ
`testPizzaCRUD()`	Valida registro en catálogo, consulta, modificación de precios y borrado.	PASÓ
`testPedidoTransaccionalCRUD()`	Comprueba transacción ACID con cabecera y líneas de detalle de pedido.	PASÓ

5.2. Salida de Ejecución de la Clase Principal (`Main.java`)

```
============================================================
  SERVICIO NACIONAL DE APRENDIZAJE (SENA) - ADSO
  SISTEMA DE GESTIÓN: PIZZERÍA BELLA ITALIA
  EVIDENCIA: GA7-220501096-AA2-EV01
  APRENDIZ: Karen Yuliana Campo | Ficha: 228118
============================================================
===> [1/4] COMPROBACIÓN DE CONEXIÓN JDBC...
  ✓ Conexión exitosa a la base de datos 'pizzeria'
  ✓ Driver: MariaDB Connector/J v3.3.3
  ✓ Motor: MariaDB v10.5.25

===> [2/4] DEMOSTRACIÓN OPERACIONES CRUD: CLIENTES...
  [CREATE] Cliente registrado con ID generado: 5 (Éxito: true)
  [READ] Datos consultados por ID: Cliente{idCliente=5, nombre='Valentina Ríos'}
  [UPDATE] Cliente actualizado con nueva dirección: Avenida Las Palmeras Casa 8
  [READ ALL] Total de clientes en el sistema: 5
  [DELETE] Cliente de prueba ID 5 eliminado con éxito.

===> [3/4] DEMOSTRACIÓN OPERACIONES CRUD: PIZZAS...
  [CREATE] Pizza registrada con ID: 7 (Pizza Cuatro Estaciones)
  [UPDATE] Precio actualizado a: $58000.00
  [DELETE] Pizza temporal ID 7 eliminada.

===> [4/4] DEMOSTRACIÓN TRANSACCIONAL: PEDIDOS Y DETALLES...
  [TRANSACTION] Pedido transaccional creado con ID: 4 (Éxito: true)
  [READ] Líneas de detalle del pedido: 2 ítems asociados
  [UPDATE] Estado actualizado a: 'Entregado'
  [DELETE] Pedido de prueba eliminado con éxito.
============================================================
  TODAS LAS OPERACIONES CRUD DE LA EVIDENCIA CUMPLIDAS 100%
============================================================
```

6. Matriz de Cumplimiento de la Rúbrica SENA (IE-GA7-220501096-AA2-EV01)

Criterio Evaluado en la Guía 7	Estado	Evidencia Técnica Aportada
1. Codifica los módulos del software teniendo en cuenta los artefactos del ciclo de software previos.	CUMPLE	Implementación basada en el diseño físico relacional `GA6-220501096-AA2-EV03.tsql`.
2. Realiza conexiones con bases de datos por medio de JDBC.	CUMPLE	Clase `ConexionBD.java` con parámetros de configuración y conexión activa en puerto 3306.
3. El código cumple con estándares de codificación (variables, métodos, clases, paquetes).	CUMPLE	Estructura en paquetes `com.sena.pizzeria.*`, clases PascalCase y métodos camelCase.
4. Dispone de funcionalidades de inserción, consulta, actualización y eliminación.	CUMPLE	CRUD implementado y comprobado en `ClienteDAO`, `PizzaDAO` y `PedidoDAO`.
5. Se crea el proyecto utilizando herramientas de versionamiento.	CUMPLE	Repositorio Git estructurado y vinculado a GitHub.
6. Archivo comprimido con la nomenclatura solicitada (ZIP).	CUMPLE	Archivo `KARENCAMPO_AA2_EV01.zip` con código fuente y enlace de repositorio.

7. Conclusiones

1. Se consolidó una solución de persistencia relacional con Java 17 y JDBC de alto rendimiento, aplicando buenas prácticas de desarrollo en capas, encapsulamiento y control de transacciones ACID.

2. La arquitectura desacoplada mediante el patrón DAO permite que los controladores web (Servlets / JSP que se abordarán en la siguiente evidencia AA2-EV02) interactúen con la base de datos de manera limpia y segura.

3. La utilización de Docker Compose facilitó la ejecución de MariaDB y la compilación Maven de forma totalmente aislada, manteniendo el sistema host limpio y ordenado.